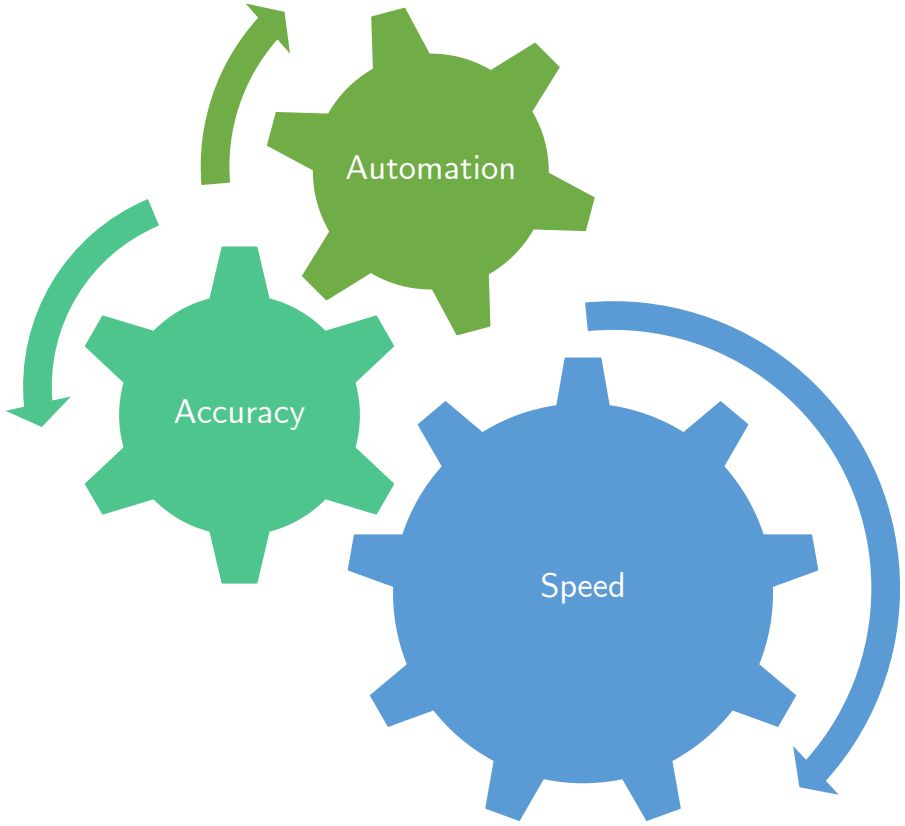
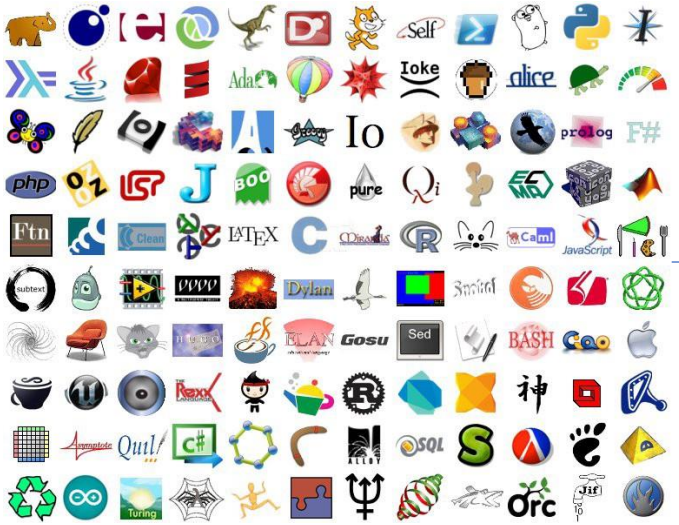
Python for Finance
(1. Mastering the Basics)

Why Coding ?



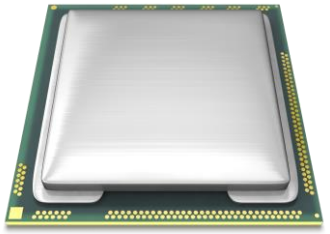
Language landscape ?

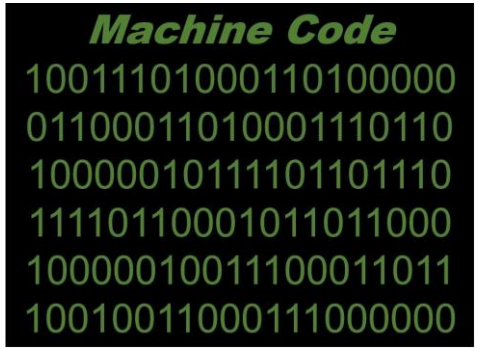
Natural Language



High level language

Compile





Machine Code

Run

Output

Why Python ?

- Open Source
- Large Community
- Plethora of free libraries
- Interpreted
- multithreading
- Elegant Code Structure

java

```java
for (int i = 0; i < 5; i++) {
  System.out.println(i);
}
```

Python

```python
for i in range(5):
    print(i)
```

The Scientific Stack

Basic Calculations

import

NumPy — Vectorized calculations (parallel processing)

pandas — Data Manipulation

matplotlib — Plotting

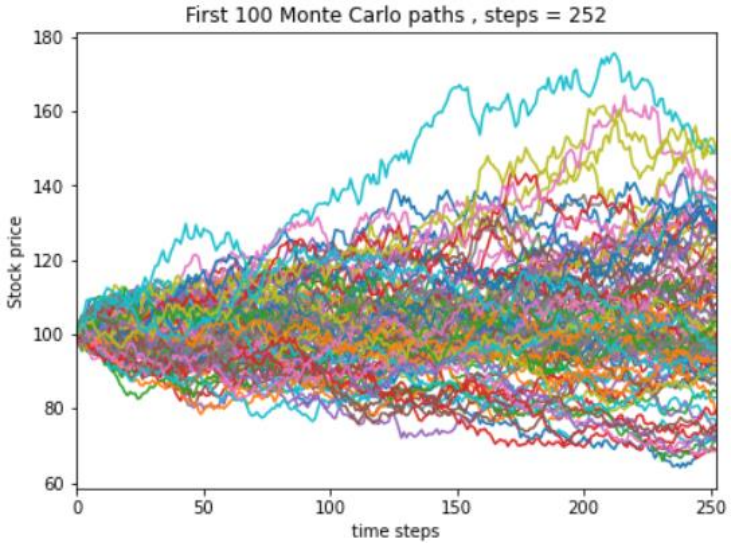
statsmodels — Statistical models, Time Series Analysis

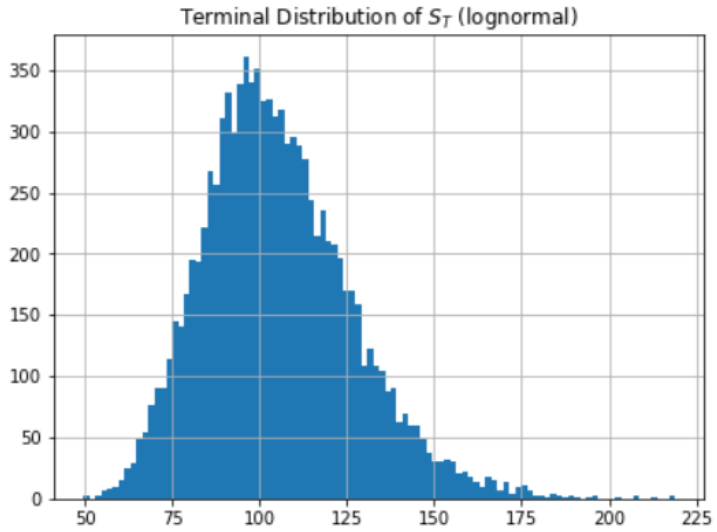
SciPy — Optimization, algebra, Calculus

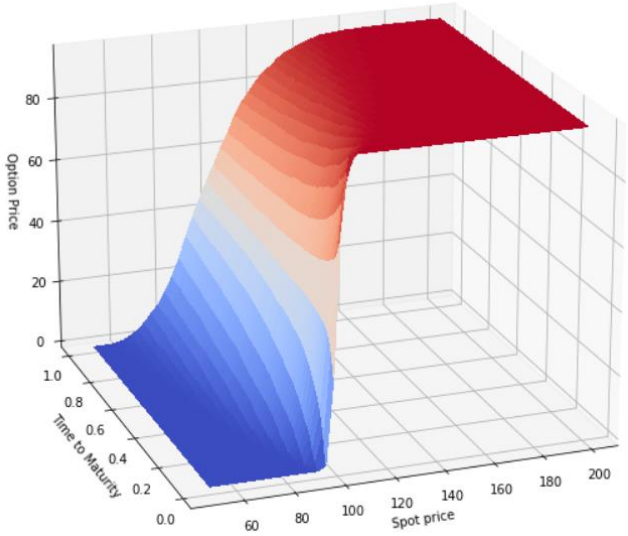
scikit learn — Machine Learning

What to expect from this course ?

- It's not a coding master class
- Focus on maths/stats modules to perform calculations in risk management
- Focus on handling financial returns data

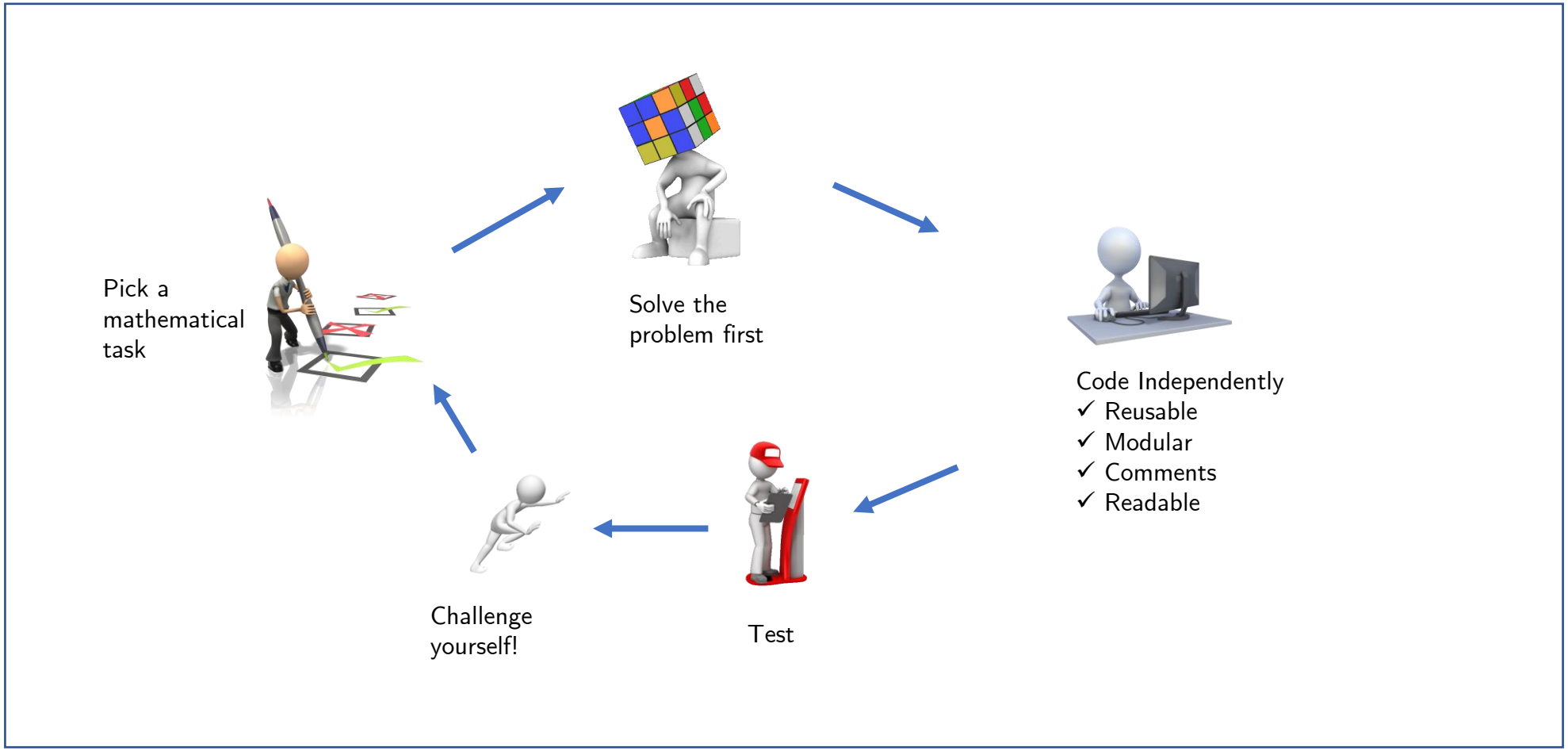






How to Learn ?

Dedicate 1 to 2 hours every day



If you are stuck ?

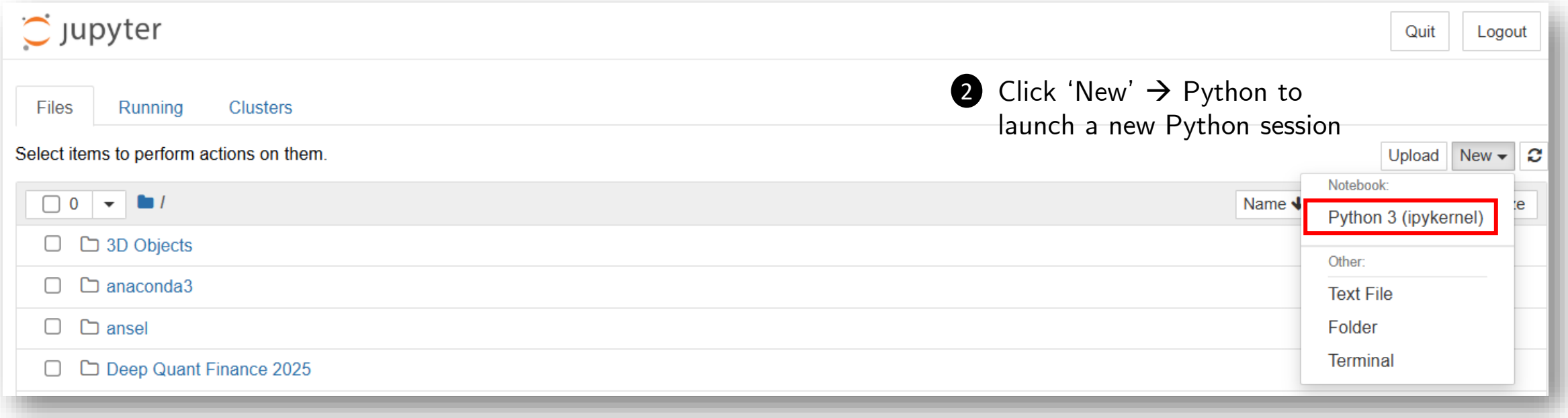
- Try to figure it out yourself by looking at the error message and experiment if needed
- Use inline help / documentation within Jupyter

stackoverflow

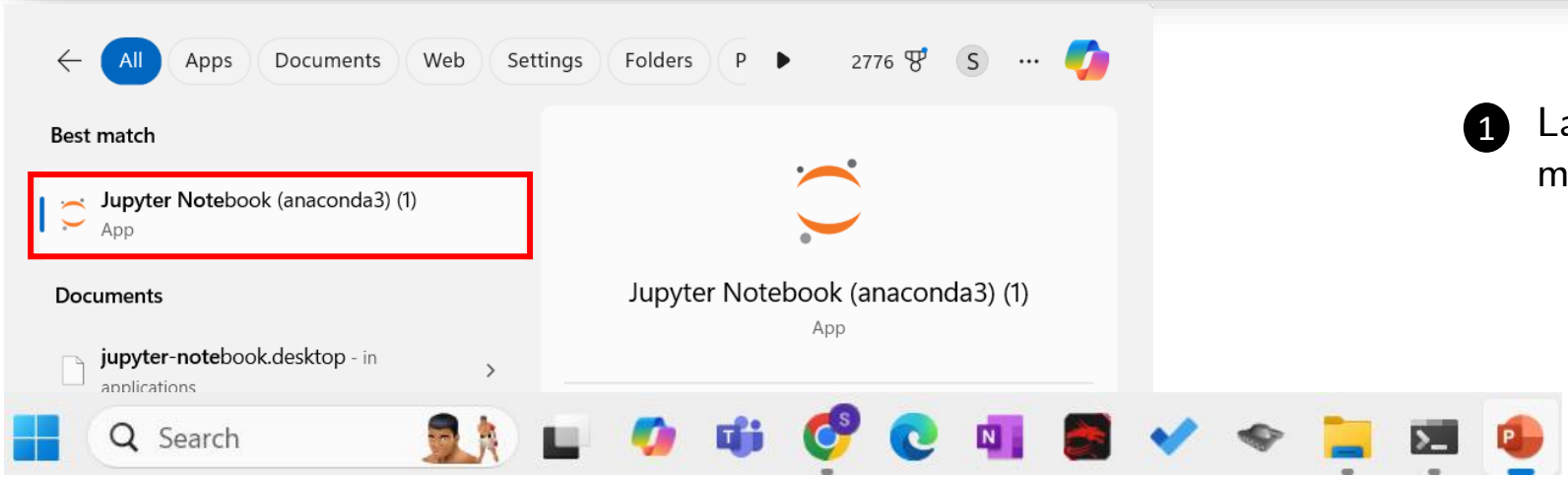
- Explore Community
- Refer to documentation

- Ask

Launch Jupyter and Python

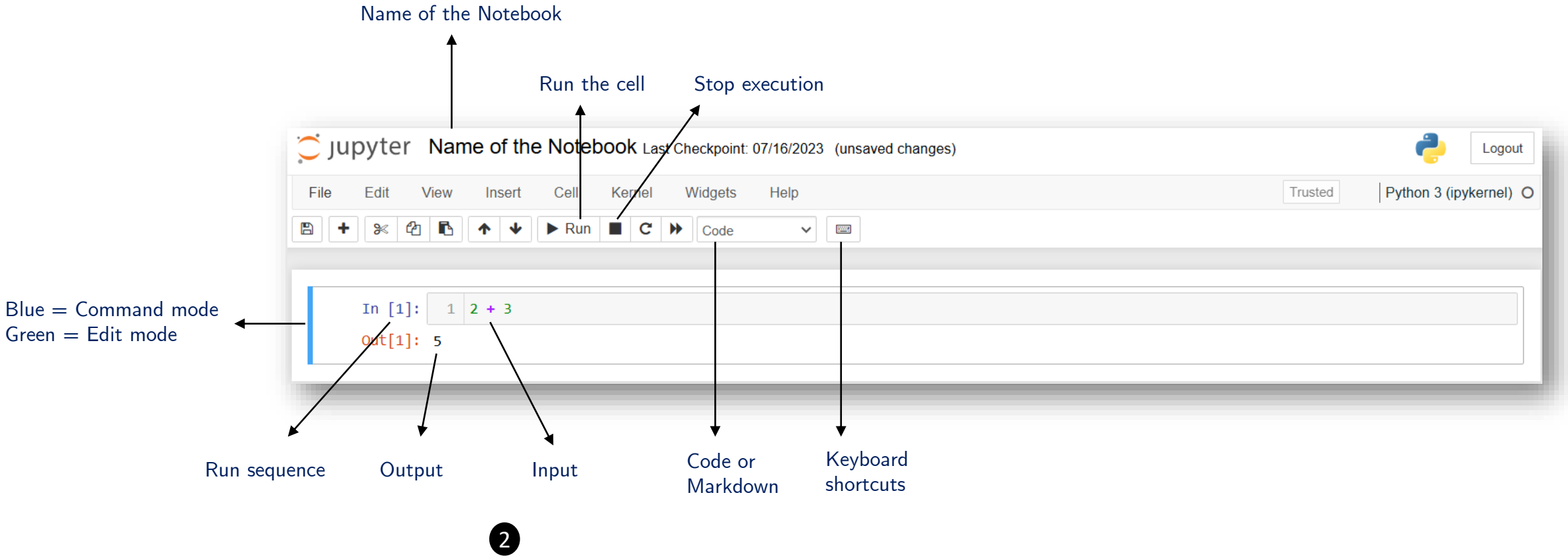


2 Click 'New' → Python to launch a new Python session



1 Launch Jupyter Anaconda from Search menu. This will open the Jupyter Home

Cell Anatomy



2

Cell Commands & hotkeys

Operation	Hotkeys
Add a new row below	B
Add a new row above	A
Delete a cell	DD
Run a cell	CTRL + ENTER
Command Mode	ESC
Edit Mode	ENTER

Markdown

- Markdown is used for adding explanatory texts, headings, urls, equations and adding images.
- Here are some common examples

Syntax	**Style**
#Text	Heading
##Text	Sub-heading
Text	Italics
Text	Bold
[Link](web address)	Hyperlink
$\Greek$	LaTex
Insert Image	Embed an image in the notebook

Python as a calculator

- Basic arithmetic operations can be performed in Python
- Here are some common examples

Operation	Output
a + b	addition
a – b	subtraction
a*b	multiplication
a/b	division
a**b	exponentiation
a//b	Floor division
a % b	modulo

Data Structure

- Python supports all the major data types that we require for routine calculations
- Here are the list of data types. Use type(variable) to fetch the data type of the variable

Data type	Example	Comments
Integer	2	
float	3.42	
Complex	2 + 3j	
String	“Hello World!”	Need to be used within single or double quotes
list	[1,2,3]	Ordered, mutable
tuple	(2, 4, 9)	Ordered, immutable
Set	{ -1, 2, 1 }	Not ordered, mutable
Dictionary	{‘ID’:201, ‘name’: ’Ryan’}	Key value pair, mutable
Boolean	True	True/False

The print function

- Print function is one of the most useful functions which is used to print results
- The print function accepts dynamic number of arguments to be printed.
- It also supports two other arguments newline, 'sep' and 'end' which serve as the separator between the arguments and the character to be printed at the end of the print statement. The default values for 'sep' and 'end' are space and newline, respectively.

```
In [13]:
1 print(2,3)
2 print('Hello')

2 3
Hello

In [15]:
1 print('Jack', 'pot', sep = '$', end = '@')
2 print('Black', 'Rock', sep = '!')

Jack$pot@Black!Rock
```

Function documentation

- **Autocomplete**
  - Type initial few letters of a function and hit TAB to autocomplete
- The inputs are called as parameters, the values that are passed are called arguments
- **Function doc string**
  - Helpful documentation about the function can be found by putting cursor inside the parenthesis and hitting SHIFT+TAB on keyboard.
  - More detailed documentation can be accessed by hitting SHIFT+TAB+TAB or putting a question mark after the function name (without the parenthesis) and running the cell.

Variable Assignment

- To assign a value to variable we use assignment operator (=), syntax : x = something
- No need to declare variable data type upfront
- Variables will automatically get their type during assignment
- If reassigned, type can change based on the value assigned
- Variables have global scope by default in the notebook

```
In [17]:  1 x = 2
          2 type(x)

Out[17]: int

In [18]:  1 x = 2 + 3j
          2 type(x)

Out[18]: complex
```

Type Casting

- Variables can be forced to be a data type using type casting
- Very useful in converting strings to integers or floating to integers etc....

```
In [29]:  1 b = "2"
          2 type(b)

Out[29]: str

In [30]:  1 int(b)

Out[30]: 2
```

Variables naming convention

- Always use meaningful names
- Follow one of the following patterns
  - HomePhoneNumber (PASCAL CASE : Beginning of all words is in CAPS)
  - home_phone_number (SNAKE CASE : Words separated by underscore)
  - homePhoneNumber (CAMEL CASE : Beginning of all words except the first in CAPS)
- Illegal names
  - Starting with a number (e.g. 1myvar)
  - Using hyphen in the name (e.g. my-var)
  - Using space in the name (my var)

Assign and self-update

- If we want to update a running variable (e.g. running sum or running product) we can use the following operators

Operation	Equivalent
x+=1	x = x+1
x*=2	x = x*2
x/=2	x = x/2
x**=2	x = x**2

Comparison Operator

- Comparison operators compare two expressions and return a Boolean

Operation	Purpose
==	equality
>	Greater than
>=	Greater than or equal to
<	Less than or equal to
<=	Less than or equal to

Logical Operator

- Logical operators are used to construct complex logic such as and/or
- The final expression evaluates to True / False (recall Truth table)

Operation	Purpose
==	equality
>	Greater than
>=	Greater than or equal to
<	Less than or equal to
<=	Less than or equal to
^	XOR

OOP primer

- Human being (Class)

A blue print, or a type

Constructor
Initialize an object

All the 7.8 billion people are instances / objects of the human class

Properties
(features / attributes)

```
{
 "name": "John",
 "age":30,
 “gender": “Male”
}
```

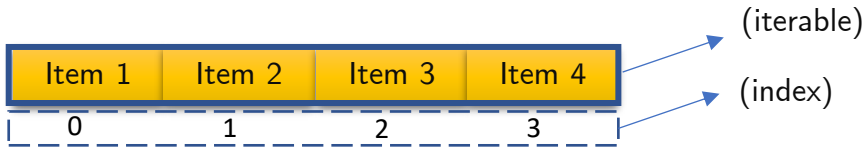
Methods
(actions a person can perform)

```
procreate()
surgery()
```

Returns a new object
(not inplace operation)

Object performing the method is altered
(in place operation)

- ✓ *Create/Instantiate* an Object
- ✓ *Read/View/Fetch/Access* an Object
- ✓ *Update/Edit* an Object
- ✓ *Delete* an Object



	string	list	tuple	set	dict
Representation	'Hello' or ""Hello"	[item1, item2, ...]	(item1, item2, ...)	{ item1, item2, ... }	{ key1:value1, key2:value2... }
Mutable	No	Yes	No	Yes	Yes
Ordered/Sequence	Yes	Yes	Yes	No	Yes
Direct Assign	s = 'Hello'	a = [1,2,3]	b = (1,2,3)	c = { 1,2,3 }	d = { 'ID':'007','Age':37 }
Constructor		list()	tuple()	set()	dict()
Fetch an item by Index	s[1] fetches item2	a[1] fetches item2	b[1] fetches item2	NA	d[1] fetches item2
Slice	s[start : stop+1 : step]	a[start : stop+1 : step]	b[start : stop+1 : step]	NA	NA
Direct Update	NA	a[1]=4	NA	NA	
Update via Method	returns a copy	in place operation	NA	mixed	In place operation
Delete	del s	del a	del b	del c	del d